UNIT 1 · TOPIC 1.3

Best Practices for Public Networks

Assignment

Instructions: 20 practice questions + 4 coding assignment(s), 39 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

1.3.a · Adversary skill levels (low-skilled vs high-skilled / zero-days); motivation

1. An attacker at a coffee shop runs a freely downloaded packet-capture script without understanding how it works. Which tier best describes this adversary? (1 pt)

- ○ A. High-skilled, because packet capture is an advanced technique
- ○ B. High-skilled, because the attack targets a public network
- ○ C. Low-skilled
- ○ D. It cannot be judged until the attacker succeeds

2. A zero-day vulnerability is dangerous primarily because: (1 pt)

- ○ A. It has been publicly announced, so many attackers know about it at once
- ○ B. It only affects systems that have not installed the latest updates
- ○ C. Antivirus signatures for it are released on day zero
- ○ D. The vendor has not yet released a patch, so defenders have no fix available

3. Which of the following motivations would lead an attacker to target a SPECIFIC high-value individual rather than sweeping widely? (Select ALL that apply.) (2 pts)

- □ A. Financial gain (bulk credential resale)
- □ B. Espionage (stealing trade secrets)
- □ C. Curiosity / learning
- □ D. Targeted corporate sabotage

(Select ALL that apply.)

4. Name ONE observable behavior that would place an attacker firmly in the high-skilled tier. (1 pt)

5. Why does the skill tier of an adversary matter for a risk assessment? (1 pt)

- ○ A. Higher-skilled attackers are easier to detect because they use more advanced tools
- ○ B. Skill tier determines the breadth and precision of possible attacks, shaping both likelihood and severity
- ○ C. Low-skilled attackers can be ignored because their tools never work
- ○ D. Skill tier matters for law enforcement but not for choosing defenses

1.3.b · Evil twin attack

6. What does BSSID stand for and why is it relevant to detecting an evil twin? (1 pt)

- ○ A. Broadband Service Signal ID, it tells you the ISP providing the connection
- ○ B. Basic Security Standard ID, it encodes the encryption type in use
- ○ C. Broadcast SSID, it is the human-readable name the user sees in the network list
- ○ D. Basic Service Set Identifier, it is the hardware MAC address of the AP radio, unique per physical device

7. A wireless scan shows two entries: both broadcast SSID 'AirportFree', one with security WPA2 and one with OPEN. Select ALL indicators that suggest an evil twin is present. (2 pts)

- □ A. The same SSID is advertised by more than one BSSID
- □ B. One entry has weaker (or no) security than the other for the same SSID
- □ C. Both entries have the same signal strength
- □ D. The network name contains the word 'Free'

(Select ALL that apply.)

8. Which attack does an evil twin primarily enable once the victim connects? (1 pt)

- ○ A. SQL injection against the victim's database
- ○ B. Man-in-the-middle, the attacker relays and can read or alter all victim traffic
- ○ C. Distributed denial-of-service against a remote server
- ○ D. Brute-force of the victim's Wi-Fi password

9. An enterprise legitimately deploys three access points all broadcasting SSID 'CorpNet' with three different BSSIDs. A fourth entry appears with SSID 'CorpNet' and a BSSID not on the approved list. What does this indicate? (1 pt)

- ○ A. Normal, any AP can join a corporate SSID
- ○ B. The network is using WPA3, which always shows extra BSSIDs
- ○ C. A misconfigured legitimate AP that should be rebooted
- ○ D. A potential evil twin; the unknown BSSID is not an approved radio

10. Name two defenses an END USER (not an administrator) can use to reduce evil-twin risk. (2 pts)

1.3.c · Jamming / denial-of-service attack

11. Which Wi-Fi access mechanism does RF jamming exploit to deny service? (1 pt)

- ○ A. WPA3 handshake, jamming corrupts the 4-way handshake
- ○ B. CSMA/CA, devices back off when they detect channel noise, including deliberate jamming
- ○ C. DHCP, jamming prevents IP address assignment
- ○ D. DNS, jamming reroutes domain lookups

12. What distinguishes RF jamming from ordinary transient interference (e.g. a nearby microwave)? (1 pt)

- ○ A. Jamming only happens on 5 GHz, while microwaves only affect 2.4 GHz
- ○ B. Jamming is louder, so any single very high sample proves an attack
- ○ C. Interference only affects one device, while jamming affects all devices
- ○ D. Jamming produces sustained, high-level noise across many consecutive samples; interference is typically brief

13. Select ALL defenses that can help a network survive or detect a jamming attack. (3 pts)

□ A. Switch to a different Wi-Fi channel not targeted by the jammer

□ B. Enable WPA3 encryption

□ C. Deploy a WIDS configured to alert on sustained noise-floor anomalies

□ D. Move from 2.4 GHz to 5 GHz or 6 GHz to use channels with shorter jammer range

(Select ALL that apply.)

14. In the coding problem, solve(samples) returns True when noise samples indicate jamming. Which rule does it apply? (1 pt)

○ A. Any single sample exceeds the threshold

○ B. The average of all samples exceeds the threshold

○ C. More than half the samples exceed the threshold

○ D. At least N consecutive samples all exceed threshold T

15. Name the metric, always a negative number, that wireless monitors use to measure signal power. (1 pt)

1.3.d · War driving; protecting yourself (verify SSIDs, avoid open networks, VPN)

16. What does a war driver primarily collect about nearby Wi-Fi networks? (1 pt)

○ A. User passwords transmitted over the networks

○ B. SSIDs, BSSIDs, security types, signal strengths, and GPS locations

○ C. The names of devices connected to each network

○ D. The browsing history of users on each network

17. Select ALL factors that a personal risk-assessment protocol should check before connecting to a public Wi-Fi network. (3 pts)

□ A. Whether the SSID has been verified with venue staff

□ B. Whether the network is open (no password)

□ C. The brand of the router hardware

□ D. Whether you have an active VPN

(Select ALL that apply.)

18. Why does SSID verification come FIRST in the personal Wi-Fi safety protocol, before VPN activation? (1 pt)

○ A. VPNs are slower than SSID checks

○ B. Venue staff can configure the VPN for you

○ C. Connecting to the wrong network (evil twin) sends traffic to an adversary before the VPN tunnel is established

○ D. Open networks cannot run VPN software

19. An attacker uses a public war-driving database to learn a corporation's SSID, then includes it in a phishing email. Which attack combination does this illustrate? (1 pt)

○ A. SQL injection combined with brute-force

○ B. Physical-layer reconnaissance combined with social engineering

○ C. Jamming combined with man-in-the-middle

○ D. Port scanning combined with evil twin

20. Name the one protection that defends you against interception even if the public Wi-Fi network is legitimate but compromised. (1 pt)

Coding Assignments (Python 3)

1.3.a · Adversary skill levels (low-skilled vs high-skilled / zero-days); motivation

3 pts · auto-graded on the site

Coding problem 1.3.a: Classify adversary skill tier

Skill: Analyze Risk · **Maps to:** 1.3.A · **Language:** Python 3 (pytest)

Problem

Security analysts use observable attacker behaviors to infer capability level. Given a profile of an adversary's tools and behaviors, classify them as either a **low-skilled** or **high-skilled** threat actor.

Function contract

Implement `solve(adversary: dict) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return `"low"` or `"high"` based on the classification rules defined in `interface.md`.

Input

A dict with exactly these boolean keys:

- `uses_zero_day`: exploits an unpublished vulnerability
- `custom_tooling`: writes or substantially modifies their own tools
- `targets_specific_victim`: selects a specific high-value target
- `off_the_shelf_only`: relies solely on public tools without modification
- `adapts_to_defenses`: changes tactics when initial attempts fail

Output

The string `"low"` or `"high"`.

Classification rules (from interface.md)

1. If `uses_zero_day` OR `custom_tooling` -> `"high"` 2. If `targets_specific_victim` AND `adapts_to_defenses` -> `"high"` 3. Otherwise -> `"low"`

Examples

```
{"uses_zero_day": True,  "custom_tooling": False, "targets_specific_victim": False,
 "off_the_shelf_only": False, "adapts_to_defenses": False}  ->  "high"

{"uses_zero_day": False, "custom_tooling": False, "targets_specific_victim": False,
 "off_the_shelf_only": True,  "adapts_to_defenses": False}  ->  "low"

{"uses_zero_day": False, "custom_tooling": False, "targets_specific_victim": True,
 "off_the_shelf_only": False, "adapts_to_defenses": True}   ->  "high"
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.3.a. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""


def solve(adversary: dict) -> str:
    """Classify an adversary as 'low' or 'high' skill based on their profile.

    Args:
        adversary: dict with boolean keys:
            uses_zero_day, custom_tooling, targets_specific_victim,
            off_the_shelf_only, adapts_to_defenses
    Returns:
        'low' or 'high' (see prompt.md and interface.md for rules)
    """
    # TODO: implement the three classification rules from interface.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

1.3.b · Evil twin attack

3 pts · auto-graded on the site

Coding problem 1.3.b: Detect evil-twin access points

Skill: Analyze Risk, Detect Attacks · **Maps to:** 1.3.B · **Language:** Python 3 (pytest)

Problem

A wireless scanner returns a list of detected access points. Each AP is a dict with keys:

- `ssid` (str), the human-readable network name
- `bssid` (str), the hardware MAC address of the radio (normalized to uppercase)
- `security` (str), one of `"OPEN"`, `"WEP"`, `"WPA"`, `"WPA2"`, `"WPA3"`
- `signal` (int), received signal strength in dBm (negative; stronger = closer to 0)

An AP is **suspicious** (potential evil twin) if ANY of the following holds:

1. **Duplicate SSID / multiple BSSIDs**, the same SSID is being broadcast by more than one BSSID in the scan. Every BSSID in such a group is flagged. 2. **Security downgrade**, an SSID appears with at least two different security levels and the AP in question does not have the strongest security level seen for that SSID. Those weaker entries are flagged.

Rules combine: an AP is flagged if either condition is true for it.

Function contract

Implement `solve(aps: list[dict]) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Return a **sorted, de-duplicated** list of suspicious BSSIDs.

Input

A list of dicts, each with keys `ssid`, `bssid`, `security`, `signal`. The list may be empty. BSSIDs are already uppercase.

Output

A sorted list of BSSIDs (strings) that are suspicious. Empty list if none.

Security ordering (weakest to strongest)

`OPEN` < `WEP` < `WPA` < `WPA2` < `WPA3`

Examples

```
# Two APs share SSID "CafeGuest" — different BSSIDs -> both flagged (rule 1)
# The OPEN entry also has weaker security (rule 2)
aps = [
    {"ssid": "CafeGuest", "bssid": "AA:BB:CC:11:22:33", "security": "WPA2", "signal": -55},
    {"ssid": "CafeGuest", "bssid": "DE:AD:BE:EF:00:01", "security": "OPEN", "signal": -42},
    {"ssid": "HotelLobby","bssid": "11:22:33:44:55:66", "security": "WPA2", "signal": -65},
]
solve(aps) -> ["AA:BB:CC:11:22:33", "DE:AD:BE:EF:00:01"]

# Empty scan
solve([]) -> []

# Single SSID, single BSSID, no anomaly
solve([{"ssid": "HomeNet", "bssid": "00:11:22:33:44:55", "security": "WPA3", "signal": -50}]) -> []
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.3.b. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List


def solve(aps: List[dict]) -> List[str]:
    """Return a sorted, de-duplicated list of suspicious BSSIDs from a wireless scan.

    An AP is suspicious if:
      1. Its SSID is advertised by more than one BSSID (duplicate-SSID rule), OR
      2. Its SSID appears with multiple security levels and this AP does not have
         the strongest security level seen for that SSID (security-downgrade rule).

    Security strength order (weakest -> strongest): OPEN < WEP < WPA < WPA2 < WPA3

    Args:
        aps: list of dicts with keys ssid, bssid, security, signal
    Returns:
        sorted list of suspicious BSSIDs (empty list if none)
    """
    # TODO: implement the two detection rules described above and in interface.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

1.3.c · Jamming / denial-of-service attack

3 pts · auto-graded on the site

Coding problem 1.3.c: Detect sustained wireless jamming

Skill: Analyze Risk · **Maps to:** 1.3.B · **Language:** Python 3 (pytest)

Problem

A wireless intrusion detection system samples the noise floor of a Wi-Fi channel once per second. Each sample is an integer noise measurement (higher = more noise). Your job is to detect whether the sample series contains a sustained jamming event.

Detection rule: a series indicates jamming if it contains a run of at least **N = 5** consecutive samples that are ALL strictly greater than **T = 80**.

These constants are fixed for this problem (see `interface.md`). A single spike, or a short burst of noise below the window, is not considered jamming.

Function contract

Implement `solve(samples: list[int]) -> bool` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Return `True` if the series contains a sustained jamming event; `False` otherwise.

Input

A list of integers (noise-floor samples). May be empty. No length limit. Values >= 0.

Output

`True` (jamming detected) or `False` (no sustained jamming).

Examples

```
[20, 25, 90, 22, 30, 18, 24]            -> False  (only one spike; run length 1 < N=5)
[15, 82, 85, 88, 91, 83, 87, 20]        -> True   (positions 1-6 all > 80; run = 6 >= N=5)
[70, 72, 75, 71, 73, 74, 76]            -> False   (none exceed T=80)
[50, 81, 82, 83, 84, 85, 86, 87, 88, 30]-> True   (positions 1-8 all > 80; run = 8 >= N=5)
[]                                       -> False  (empty input)
```

Constraints

- T = 80 (strictly greater than), N = 5 (minimum consecutive samples above T).
- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.3.c. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List

# Fixed detection constants (do not change)
THRESHOLD = 80   # noise floor threshold (strictly greater than)
WINDOW = 5       # minimum consecutive samples above threshold to flag jamming
```

```
def solve(samples: List[int]) -> bool:
    """Return True if the sample series contains a sustained jamming event.

    A series is flagged if it contains a run of at least WINDOW consecutive
    samples that are ALL strictly greater than THRESHOLD.

    Args:
        samples: list of integer noise-floor measurements
    Returns:
        True if jamming detected, False otherwise
    """
    # TODO: scan samples and detect a run of WINDOW consecutive values > THRESHOLD
    raise NotImplementedError("implement solve() — see prompt.md")
```

1.3.d · War driving; protecting yourself (verify SSIDs, avoid open networks, VPN)

3 pts · auto-graded on the site

Coding problem 1.3.d: Public Wi-Fi risk flagger

Skill: Mitigate Risk · **Maps to:** 1.3.B, 1.3.C · **Language:** Python 3 (pytest)

Problem

You are building the risk-assessment routine a network-policy tool runs before letting a device connect to a public Wi-Fi network. Given a description of one network and the user's current posture, return the **risk flags** that apply, so the user (or an administrator) can make an informed decision. This is exactly the three-step personal protocol from the lesson, expressed as code: verify the SSID, avoid open networks, always use a VPN.

The function is pure and deterministic, no network, no I/O.

Function contract

Implement `solve(network: dict) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`network`, a `dict` describing one network and the user's posture toward it. Recognized keys:

Key	Type	Meaning
`"encryption"`	`str`	Wi-Fi security type, e.g. `"open"`, `"wpa2"`, `"wpa3"`. Case-insensitive. Missing / `None` / `""` is treated as open (worst case).
`"verified"`	`bool`	`True` if the SSID was confirmed with venue staff / an official source. Missing is treated as `False`.
`"vpn"`	`bool`	`True` if a VPN tunnel is active on the device. Missing is treated as `False`.

Example:

```
{"ssid": "AirportFreeWifi", "encryption": "open", "verified": False, "vpn": False}
```

Output

A **sorted, de-duplicated** `list[str]` of the risk flags that apply, drawn from:

- `"open-network"`: the network is open (no Wi-Fi-layer encryption).
- `"unverified-ssid"`: the SSID has not been confirmed with an authoritative source.
- `"no-vpn"`: no VPN is protecting the user's traffic.

Return `[]` when none apply (a verified, encrypted network used over a VPN).

Examples

```
{"encryption": "open", "verified": False, "vpn": False}   -> ["no-vpn", "open-network", "unverified-ssid"]
{"encryption": "wpa2", "verified": True,  "vpn": True}     -> []
{"encryption": "wpa2", "verified": True,  "vpn": False}    -> ["no-vpn"]
{"encryption": "WPA3", "verified": False, "vpn": True}     -> ["unverified-ssid"]
{}                                                          -> ["no-vpn", "open-network", "unverified-ssid"]
```

Constraints

- Deterministic, offline, standard library only.
- Matching of `encryption` is case-insensitive.
- Output always sorted ascending and de-duplicated.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.3.d. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List


def solve(network: dict) -> List[str]:
    """Return the sorted, de-duplicated risk flags for one network description.

    Flags: "open-network", "unverified-ssid", "no-vpn".
    See prompt.md and interface.md for the rules.

    Args:
        network: dict with optional keys "encryption" (str), "verified" (bool), "vpn" (bool).
    Returns:
        sorted list of the risk-flag strings that apply ([] if none).
    """
    # TODO: flag "open-network" when encryption is open/missing,
    #       "unverified-ssid" when not verified, "no-vpn" when no VPN active.
    #       Return the flags sorted and de-duplicated.
    raise NotImplementedError("implement solve() — see prompt.md")
```